

STL 容器三大分类 + 核心常用容器

一、序列式容器

(线性排列, 元素有顺序)

最常用、最简单, 你刚手写的就是这一类

1. `vector` 🏆 最重要 (官方版的 `myArray`)

- **本质:** 动态数组
- **特点:** 尾插尾删极快、支持随机访问 (`[]`)、中间插入慢
- **用途:** 存任意数据, 替代普通数组

```
vector<int> v;  
v.push_back(10); // 尾插  
v[0];           // 下标访问
```

2. `string`

- **本质:** 字符数组
- **特点:** 专门存字符串, 拼接、查找、替换超方便
- **用途:** 处理姓名、文本等字符串

```
string s = "hello";  
s += " world";
```

3. `deque` 双端队列

- **本质:** 双口动态数组
- **特点:** 头插、尾插都快, 支持随机访问
- **用途:** 需要两头增删数据的场景

4. `list` 双向链表

- **本质:** 链表
- **特点:** 任意位置插入 / 删除都快, 不支持 `[]` 随机访问
- **用途:** 频繁在中间增删数据

二、关联式容器

自动排序、查找速度极快, 键值对存储, 适合快速查找、去重、排序。

STL 关联式容器分两大类：

1. 有序关联式容器（底层红黑树）：`map / set`
2. 无序关联式容器（底层哈希表）：`unordered_map / unordered_set`

5. `set` 集合

- 特点：元素唯一、自动排序、查找超快
- 用途：去重、排序、快速查找

6. `map` 键值对（字典）

- 特点：`key-value` 结构，key 唯一、自动排序
- 用途：存对应关系（比如 姓名 -> 年龄，学号 -> 姓名）

```
map<string, int> m;  
m["张三"] = 18;
```

7. `unordered_map` 无序字典(哈希映射)

- 存储键值对 `key-value`（和 `map` 一模一样）
- key 唯一不重复（和 `map` 一模一样）
- 不自动排序！（和 `map` 最大区别）
- 查找 / 插入 / 删除速度极快（刷题永远优先用它）

8. `unordered_set` 无序集合 (哈希集合)

底层是哈希表的无序集合，和 `set` 规则一致，但更快、无序。

1. 只存单个值（不是键值对，和 `set` 一样）
2. 元素唯一，自动去重（重复插入会被忽略）
3. 无序、查找 / 插入速度极快 $O(1)$ （刷题首选）
4. 不支持 `[]`，不能修改元素

三、容器适配器

容器适配器 = 不是真正的容器，而是「套壳改装」的工具

它没有自己独立的底层数据结构，而是复用你已经学过的 `deque/vector/list` 这些底层容器，限制接口、定制规则，改造成新的工具。

7. `stack` 栈

- 规则：先进后出
- 用途：撤销、括号匹配、函数调用栈

8. `queue` 队列

- 规则：先进先出
- 用途：排队、消息队列

9. `priority_queue` 优先队列（堆）

- 规则：不遵循先进先出，队头永远是优先级最高的元素；默认是大顶堆（最大值优先），可修改为小顶堆（最小值优先）
 - 默认底层容器：`vector`
 - 用途：TopK 问题、贪心算法、合并 K 个升序链表、快速获取最值
 - 核心特点：插入元素后自动排序，是算法题中堆的标准实现
-

四、LeetCode Hot100 刷题必背 STL 优先级

1. `vector`（动态数组）★★★★★

刷题使用率第一，所有题目基础容器

- 核心用途：存储数据、动态数组、模拟所有线性结构
- 必用场景：遍历、排序、二分、数组类所有题目

2. `unordered_map`（无序哈希映射）★★★★★

哈希表刷题神器，Hot100 高频题核心

- 核心用途：键值对映射、快速查找、统计次数
- 必用场景：两数之和、字母异位词、求和类题目

3. `unordered_set`（无序哈希集合）★★★★★

最快的去重 / 查重工具，和哈希映射绑定使用

- 核心用途：判断元素是否存在、数组 / 字符串去重
- 必用场景：重复元素、找不同、哈希查重

4. `string` (字符串) ★★★★★

字符串类题目专属容器，高频中的高频

- 核心用途：字符串处理、翻转、匹配、子串
- 必用场景：回文串、有效括号、字符串分割

5. `stack` (栈) ★★★★★

经典结构，括号 / 单调栈题型必考

- 核心用途：先进后出、括号匹配、逆序处理
- 必用场景：有效的括号、柱状图最大矩形

6. `priority_queue` (优先队列 / 堆) ★★★★★

TopK 问题、贪心算法**必考**

- 核心用途：快速取最大 / 最小值、堆排序
- 必用场景：前 K 个高频元素、数据流中位数

7. `queue` (队列) ★★★★★

搜索类题目标配

- 核心用途：先进先出、层级遍历
- 必用场景：二叉树层序遍历、广度优先搜索 (BFS)

8. `map` / `set` (有序映射 / 集合) ★★

极低频率使用，仅题目要求**有序**时才用

- 核心用途：自动排序的键值对 / 集合
- 适用场景：极少数需要有序的题目 (Hot100 几乎不用)

9. `deque` (双端队列) ★

进阶使用，单调队列专用

- 核心用途：首尾增删、滑动窗口
- 适用场景：滑动窗口最大值 (难题)

10. `list` (双向链表) ✖

刷题完全不用，直接忽略即可

- LeetCode Hot100 无任何题目需要使用
-

一、vector容器

`vector` 是 STL 最常用的动态数组容器，它底层帮你封装好了：内存管理、深拷贝、尾插尾删、下标访问、扩容..... 你不用写一行底层代码，直接调用接口就行。

一、核心基础

1. 头文件

```
#include <vector> // 必须包含
using namespace std;
```

2. 本质

- 动态数组（自动扩容，不用管堆区内存）
- 模板类 `vector<T>`，`T` 可以是 `int/string/Person` 任意类型
- 自动实现深拷贝，完全避免浅拷贝崩溃

二、vector 常用构造函数（创建对象）

和你 `myArray` 的构造函数一一对应：

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // 1. 空构造（最常用）
    vector<int> v1;

    // 2. 指定容量构造
    vector<int> v2(10); // 容量10，初始值默认为0

    // 3. n个相同值构造
    vector<int> v3(5, 100); // 5个100

    // 4. 拷贝构造
    vector<int> v4(v3);

    // 5. 初始化列表构造（C++11）
    vector<int> v5 = {1,2,3,4,5};
}
```

三、赋值操作

```
vector<int> v1;
vector<int> v2;

// 1. = 赋值（对应你重载的 operator=, 自动深拷贝）
v1 = v2;

// 2. assign 赋值
v1.assign(5, 100); // 5个100
v1.assign(v2.begin(), v2.end());
```

四、容量 & 大小 & 预留

这是你最熟悉的概念，直接对应：

函数	作用	对应你的 myArry
<code>empty()</code>	判断是否为空（空返回 true）	-
<code>size()</code>	获取当前元素个数	<code>getSize()</code>
<code>capacity()</code>	获取数组容量	<code>getCapacity()</code>
<code>resize(num)</code>	重新指定大小，多删少补默认值	-
<code>reserve(num)</code>	提前预留容量（预留位置不初始化，元素不可访问）	-

示例：

```
vector<int> v(10);
cout << v.size() << endl; // 10
cout << v.capacity() << endl; // 10
cout << v.empty() << endl; // 0（非空）
```

• 预留空间可以免去vector在存入数据时频繁扩容：

```
// 1. 创建一个空的int型vector
vector<int> v;

// 2. 【核心】预留空间：提前分配容量为100000
// 只分配内存，不创建元素，size依然是0
v.reserve(10000);

// 3. 计数器：统计vector扩容的次数
int num = 0;

// 4. 指针：保存vector底层数组的首地址
int* p = NULL;

// 5. 循环插入100000个数据
for (int i = 0; i < 100000; i++) {
    v.push_back(i); // 尾插数据
}
```

```
// 6. 判断: vector数组的首地址是否发生变化
// 地址变了 = 触发了扩容
if (p != &v[0]) {
    p = &v[0];    // 更新指针, 保存新的首地址
    num++;        // 扩容次数+1
}
```

• 核心原理 (必懂!)

1. 为什么 `&v[0]` 能判断扩容?

`vector` 底层是堆区数组:

- 数组满了 → 自动扩容
- 扩容 = 重新开辟一块更大的堆内存 → 拷贝旧数据 → 释放旧内存
- 内存地址变了 → `&v[0]` (首地址) 就会变

2. `reserve(100000)` 到底做了什么?

`reserve(容量)` = 提前一次性开好足够大的内存

- 只修改 `capacity` (容量)
- 不修改 `size` (元素个数)
- 不初始化元素
- 插入 100000 个数据时, 容量永远够用, 不会触发任何扩容

四、运行结果对比 (最直观)

情况 1: 代码中加了 `v.reserve(100000)`

✓ 运行结果: `num:1`

- 只初始化了一次地址
- 0 次扩容

情况 2: 注释掉 `v.reserve(100000)`

✗ 运行结果: `num:18` 左右

- `vector` 会频繁自动扩容 (每次扩 1.5 倍 / 2 倍)
 - 扩容 17~18 次, 大量拷贝数据, 效率极低
-

五、数据存取

```
vector<int> v = {10,20,30};

// 1. [] 访问（不检查越界，对应你重载的 operator[]）
cout << v[0] << endl;

// 2. at() 访问（会检查越界，抛异常，更安全）
cout << v.at(1) << endl;

// 3. 获取第一个元素
cout << v.front() << endl;

// 4. 获取最后一个元素
cout << v.back() << endl;
```

六、插入 & 删除

这是 vector 最常用的功能.

```
vector<int> v;

// 1. 尾插法 ✨ 最常用（对应你 push_back）
v.push_back(10);
v.push_back(20);

// 2. 尾删法 ✨ 最常用（对应你 pop_back）
v.pop_back();

// 3. 指定位置插入（迭代器位置）
v.insert(v.begin(), 100); // 开头插入100

// 4. 删除指定位置元素
v.erase(v.begin()); // 删除第一个元素

// 5. 清空所有元素
v.clear();
```

七、vector 遍历方式（和 string 完全一样，3 种）

以 `vector<int> v = {1,2,3,4,5};` 为例

方式 1：普通 for + 下标（你最熟，最简单）

```
for (int i = 0; i < v.size(); i++) {
    cout << v[i] << " ";
}
```


方式 2：迭代器遍历（STL 通用正统写法）

```
// 迭代器类型: vector<int>::iterator
for (vector<int>::iterator it = v.begin(); it != v.end(); it++) {
    cout << *it << " ";
}
```

方式 3：范围 for（C++11，最简洁）

```
vector<T> v_name;
for (T num : v) {
    cout << num << " ";
}
```

八、存放自定义数据类型

vector 可以直接存你的 `Person` 对象，完全不用改底层：

```
class Person {
public:
    string m_Name;
    int m_Age;
    Person() {}
    Person(string name, int age) : m_Name(name), m_Age(age) {}
};

int main() {
    vector<Person> v;
    v.push_back(Person("孙悟空", 30));
    v.push_back(Person("韩信", 20));

    // 遍历
    for (Person p : v) {
        cout << p.m_Name << " " << p.m_Age << endl;
    }
}
```

九、vector 嵌套（二维数组）

1.概念

二维 `vector`：多行数字（表格 / 矩阵），就是把一维 `vector` 装进另一个 `vector` 里

```
vector<vector<int>> v; // 相当于几行几列的表格
```

```
// 二维vector: vector里放vector
vector<vector<int>> v;

vector<int> v1;
vector<int> v2;
v.push_back(v1);
v.push_back(v2);
```

2.基础语法（必记）

（1）定义二维 vector

```
// 定义空的二维数组（最常用）
vector<vector<int>> v;

// 定义 n行 m列 的二维数组（指定大小，直接用）
vector<vector<int>> v(n, vector<int>(m));
```

- 外层 vector 管行
- 内层 vector 管列

（2）初始化赋值（直接填数）

```
// 直接创建一个 2行3列 的数组
vector<vector<int>> v = {
    {1, 2, 3}, // 第0行
    {4, 5, 6}  // 第1行
};
```

3.最关键：访问元素

和一维数组几乎一样，格式：`v[行号][列号]`

下标从 0 开始（和你之前学的完全一致）

示例：

```
v[0][1] = 2; // 第0行第1列 → 2
v[1][2] = 6; // 第1行第2列 → 6
```

4.遍历二维数组（双层循环）

外层循环管行，内层循环管列

```
// 遍历所有元素
for(int i=0; i<v.size(); i++){           // i: 行号
    for(int j=0; j<v[i].size(); j++){ // j: 列号
        cout << v[i][j] << " ";
    }
    cout << endl; // 每行结束换行
}
```

5.输入数据（和一维用法一致）

```
int n, m;
cin >> n >> m; // 输入行数n，列数m
vector<vector<int>>> v(n, vector<int>(m)); // 创建n行m列数组

// 循环输入
for(int i=0; i<n; i++){
    for(int j=0; j<m; j++){
        cin >> v[i][j];
    }
}
```

十、vector互换容器：收缩内存

- v1.swap(v2)

```
vector<int>v1;
vector<int>v2;
v1.swap(v2);
```

作用：收缩内存，这是 `vector` 最经典的内存优化技巧。

- **核心前提：vector 的容量 (capacity) 只增不减**

vector 有两个核心属性：

1. `size`：实际存了多少元素（你正在用的房间）
2. `capacity`：总容量（开发商给你盖的整栋楼）

- **关键规则：**

vector 只会**自动扩容**（不够用了就盖更大的楼），

不会自动缩容（你用不完，楼也不会拆，空着浪费）！

• `resize()`只能扩容不能缩容

```
vector<int> v;  
// 插入10万条数据  
for (int i = 0; i < 100000; i++) {  
    v.push_back(i);  
}  
// 此时: size=100000, capacity≈130000左右 (vector自动扩容的结果)  
  
v.resize(3);  
// 重点: resize只把【大小】改成3  
// 【容量】还是 13万!!!  
// 相当于: 一栋13万平的楼, 你只住3平米, 剩下全空着 → **严重内存浪费**
```

这就是**必须收缩内存**的原因:

把多余的、空着的内存**还给系统**, 不浪费空间。

• 核心代码拆解: `vector<int>(v).swap(v);`

这一行是**收缩内存的神级写法**, 我拆成 3 步给你看:

第一步: `vector<int>(v)`

创建一个**匿名对象** (没有名字的临时对象)

- 用 `v` 拷贝构造
- 这个匿名对象**只拷贝 `v` 里实际的 3 个元素**
- 它的 `size = 3`, `capacity = 3` (刚好够用, 不浪费)

第二步: `.swap(v)`

调用 `swap` 互换容器

- 把 **匿名对象的小内存** 和 **`v` 的大内存** 彻底交换
- 交换后:
 - `v` 指向小内存 (`size=3`, `capacity=3`)
 - 匿名对象指向原来的大内存 (13 万容量)

第三步: 匿名对象自动销毁

这行代码执行完, **匿名对象生命周期结束**, 系统自动释放它持有的**原来 `v` 的大内存!**

最终结果

```
// 收缩内存前
v的容量为: 138240 （很大）
v的大小为: 3

// 收缩内存后
v的容量为: 3 （和大小一致）
v的大小为: 3
```

十、超级重点总结（只记这些就够）

1. **vector = 官方版 myArray**，用法几乎一模一样
2. 必用接口：
 - `push_back` 尾插
 - `pop_back` 尾删
 - `[]` / `at` 访问元素
 - `size()` 大小 / `capacity()` 容量
3. **自动深拷贝**，不用写构造 / 析构 / 赋值重载
4. **三种遍历**：下标、迭代器、范围 `for`
5. 支持**所有数据类型**：int、string、自定义对象

二、string 容器

`string` 是 STL 专门用来处理字符串的容器，完全替代 C 语言的 `char*`，也是你写代码最常用的容器。

它的核心优势：

1. 自动管理内存（不用 `new/delete`，不用怕浅拷贝）
2. 封装了大量现成方法（拼接、查找、替换、删除...）
3. 用法简单，像用普通变量一样方便

一、必备基础

1. 头文件

```
#include <string>    // 必须加
using namespace std;
```

2. 本质

`string` 是一个类，底层封装了字符数组，自动做深拷贝，你完全不用管内存！

二、构造函数（创建字符串）

常用 4 种创建方式，记住前 2 个就够：

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // 1. 创建空字符串（最常用）
    string s1;

    // 2. 使用字符串常量初始化
    string s2("hello world");

    // 3. 拷贝构造（用已有字符串创建）
    string s3(s2);

    // 4. 用 n 个相同字符创建
    string s4(5, 'a'); // "aaaaa"

    return 0;
}
```

三、赋值操作（改字符串）

= 直接用，最简单！

```
string s;
s = "hello";    // 直接赋值
s = 'a';        // 赋值单个字符
s = s2;         // 赋值另一个 string

// assign 赋值（很少用）
s.assign("hello");
s.assign(5, 'b'); // "bbbbbb"
```

四、3. 字符串拼接（最常用！）

直接用 `+`，比 C 语言的 `strcat` 方便 100 倍！

```
string s1 = "hello ";
string s2 = "world";

// 1. + 拼接（推荐）
string s3 = s1 + s2;    // "hello world"
s1 += "张三";           // 追加字符串
s1 += '!';              // 追加字符

// 2. append 拼接（等价于 +=）
s1.append("abc");
```

五、查找与替换

查找 (find)

`int` 第一次出现的下标 `=s.find("要查找的字符串")`

默认从头开始寻找

`int` 第一次出现的下标 `=s.find("要查找的字符串", 开始查找的下标)`

`rfind`:从右向左搜寻。

查找字符 / 字符串，返回**第一次出现的下标**

找不到返回 `-1` (`string::npos`)

```
string s = "abcdefabc";

// 从下标0开始查找 "abc"
int pos = s.find("abc"); // 0
// 从下标1开始查找
pos = s.find("abc", 1); // 6

// 找不到返回 -1
if (s.find("xyz") == -1) {
    cout << "没找到" << endl;
}
```

替换 (replace)

`s.replace(从这个下标位置开始替换, 替换多少个字符, "用来替换的字符串")`

```
// 从下标0开始, 替换3个字符为 "111"
s.replace(0, 3, "111"); // "111defabc"
```

六、字符存取（访问单个字符）

两种方式，`[]` 最常用

```
string s = "hello";

// 方式1: [] （不检查越界，写错直接崩溃）
cout << s[0] << endl; // 'h'
s[0] = 'H';           // 修改

// 方式2: at() （检查越界，抛异常，更安全）
cout << s.at(1) << endl; // 'e'
```

七、插入与删除

- `insert`(要插入的位置, “要插入的字符串”)
- `erase`(要删除的位置, 要删除的字符个数)
- `clear()`

```
string s = "hello";

// 插入: 在下标0插入 "123"
s.insert(0, "123"); // "123hello"

// 删除: 从下标0删3个字符
s.erase(0, 3); // "hello"
//删除末尾字符
s.erase(str.size() - 1);

// 清空字符串
s.clear();
```

- `pop_back()`: 删除最后一个字符 (C++11 及以上)

```
int main() {
    string str = "Hello world";

    // 关键: 必须先判断字符串非空, 防止空字符串删最后一个元素导致未定义行为
    if (!str.empty()) {
        str.pop_back(); // 删除最后一个字符
    }

    cout << str << endl; // 输出: Hello worl
    return 0;
}
```

- `resize(n)`: 可以直接将字符串长度缩减为 `n`, 缩减长度时会自动删除末尾多余字符


```
int main() {
    string str = "Hello world";

    if (!str.empty()) {
        // 长度减1，直接删除最后一个字符
        str.resize(str.size() - 1);
    }

    cout << str << endl; // 输出: Hello worl
    return 0;
}
```

八、获取子串（截取字符串）

- `s.substr()` (从这个下标开始，截取多少个字符)

```
string s = "abcdef";
// 从下标2开始，截取3个字符
string sub = s.substr(2, 3); // "cde"
```

九、常用工具函数

- 获取字符串长度 `size()` / `length()`，返回 `int` 类型的长度
- 判断是否为空 `empty()`，返回真/假
- 获取首字符 `front()`、尾字符 `back()`

```
string s = "hello";

// 1. 获取字符串长度（两个函数完全一样）
int len = s.size();
int len2 = s.length();

// 2. 判断是否为空
if (s.empty()) { ... }

// 3. 获取首字符/尾字符
s.front();
s.back();
```

十、字符串遍历（3 种方式）

方式 1：普通 for 循环（最常用）

```
for (int i = 0; i < s.size(); i++) {
    cout << s[i] << " ";
}
```

方式 2：范围 for（C++11，最简单）

```
for (char c : s) {
    cout << c << " ";
}
```

方式 3：迭代器（STL通用）

```
for (string::iterator it = s.begin(); it != s.end(); it++) {
    cout << *it << " ";
}
cout << endl;
```

十一、string 对比 char*（为什么要用 string？）

特性	char* (C 语言)	string (C++)
内存管理	手动 new/delete	自动管理
拼接	麻烦 (strcat)	直接 +
拷贝	浅拷贝，易崩溃	自动深拷贝
长度获取	strlen	size()
易用性	差	极高

三、deque 双端队列

你刚学会了 vector（单端动态数组），deque 就是 vector 的升级版 —— 双端动态数组！

它完美解决了 vector 头部插入 / 删除极慢的缺点，是 STL 中最实用的容器之一。

一、核心先搞懂

1. 什么是 deque?

全称：**双端队列 (Double-ended Queue)**

- ✓ **本质**：分段连续的双端动态数组
- ✓ **最大优势**：头插、头删、尾插、尾删 效率都极高
- ✓ **通用能力**：支持 `[]` 随机访问（和 `vector` 一模一样）
- ✓ **内存**：STL 自动管理，无需手写深拷贝 / 析构

2. 对比 vector（一眼区分）

容器	插入位置	随机访问	适用场景
<code>vector</code>	只有尾插 / 尾删快	支持	只需尾部操作
<code>deque</code>	头、尾操作都快	支持	需要两头增删

3. 头文件

```
#include <deque> // 必须包含
using namespace std;
```

二、构造函数（和 vector 完全一样，直接套）

你会 `vector` 的构造，就会 `deque`：

```
// 1. 空容器
deque<int> dq1;

// 2. 指定大小
deque<int> dq2(5);

// 3. 指定大小+初始值
deque<int> dq3(5, 100);

// 4. 拷贝构造
deque<int> dq4(dq3);

// 5. 初始化列表
deque<int> dq5 = {1,2,3,4,5};
```

三、🔥 deque 核心功能：双端操作（vector 没有！）

这是 `deque` 最独特、最常用的 API，**必须记住**：

```
deque<int> dq;
```

```
// 1. 尾插（和 vector 一样）
dq.push_back(10);
dq.push_back(20);

// 2. 尾删（和 vector 一样）
dq.pop_back();

// =====
// 🔥 重点：deque 独有！头部操作
// =====
// 3. 头插（最常用）
dq.push_front(1); // 头部插入 1
dq.push_front(2); // 头部插入 2

// 4. 头删
dq.pop_front(); // 删除第一个元素
```

四、数据存取（和 vector 完全一样）

```
deque<int> dq = {10,20,30};

// 1. [] 随机访问
cout << dq[0] << endl;

// 2. at() 访问
cout << dq.at(1) << endl;

// 3. 获取首尾元素
cout << dq.front() << endl; // 第一个
cout << dq.back() << endl;  // 最后一个
```

五、遍历方式（和 vector/string 完全相同，不用新学！）

三种遍历通用，直接套模板：

方式 1：普通 for + 下标

```
for (int i = 0; i < dq.size(); i++) {
    cout << dq[i] << " ";
}
```

方式 2：迭代器（STL 通用）

```
for (deque<int>::iterator it = dq.begin(); it != dq.end(); it++) {
    cout << *it << " ";
}
```

方式 3：范围 for（最简）

```
for (int num : dq) {  
    cout << num << " ";  
}
```

六、插入 & 删除（通用）

```
deque<int> dq = {1,2,3};  
  
// 指定位置插入  
dq.insert(dq.begin() + 1, 100); // 下标1插入100  
  
// 指定位置删除  
dq.erase(dq.begin() + 1); // 删除下标1元素  
  
// 清空容器  
dq.clear();
```

七、常用大小操作（和 vector 一致）

```
dq.size(); // 获取元素个数  
dq.empty(); // 判断是否为空  
dq.resize(10); // 重新指定大小
```

八、超简总结（背会这 3 点就够）

1. deque = 双端 vector
2. 独有核心：push_front（头插）、pop_front（头删）
3. 用法：除了头插 / 头删，其余和 vector 一模一样

四、list 双端列表

list 是 STL 中的双向链表容器

一、核心本质（一句话记住）

list = 双向循环链表

- 底层**不是连续内存**，每个元素独立存储，通过指针相连
- 不支持 `[]` 下标访问！
- **任意位置插入 / 删除数据极快** (秒杀 vector/deque)

二、list 对比 vector（你最关心的区别）

特性	vector（动态数组）	list（双向链表）
内存结构	连续内存	分散内存，链式存储
随机访问（ <code>[]</code> ）	✅ 支持	❌ 不支持
头部 / 中间插入删除	❌ 很慢（要挪数据）	✅ 极快
尾部插入删除	✅ 快	✅ 快
迭代器失效	扩容 / 插入易失效	插入删除不失效
占用内存	小	大（存指针）

三、基础用法

1. 头文件

```
#include <list> // 必须包含
using namespace std;
```

2. 构造函数（和 vector 完全一样）

```
// 1. 空链表
list<int> lt1;

// 2. n个默认值
list<int> lt2(5);

// 3. n个指定值
list<int> lt3(5, 100);

// 4. 拷贝构造
list<int> lt4(lt3);
```

四、🔥 list 最常用核心 API

1. 头尾增删（和 deque 一模一样）

```
list<int> lt;

// 尾插
lt.push_back(10);
```

```
// 尾删
lt.pop_back();

// 头插
lt.push_front(20);
// 头删
lt.pop_front();
```

2. ❌ 禁止使用: `lt[0]`

`list` 没有重载 `[]`, 不能下标访问!!!

这是新手最容易踩的坑!

3. 访问元素 (只能访问首尾)

```
// 第一个元素
lt.front();
// 最后一个元素
lt.back();
```

4. 插入 / 删除 (任意位置, 效率极高!)

必须用迭代器指定位置:

```
list<int> lt = {1,2,3};

// 迭代器指向开头
list<int>::iterator it = lt.begin();

// 在迭代器位置插入 100
lt.insert(it, 100);

// 删除迭代器位置元素
lt.erase(it);

// 清空所有
lt.clear();
```

• 插入 / 删除中间元素

操作	函数	说明
中间插入	<code>insert(迭代器位置, 数值)</code>	在迭代器指向的位置前面插入
中间删除	<code>erase(迭代器位置)</code>	删除迭代器指向的元素

以链表 `10 20 30 40` 为例: 我们要在 **20 和 30 中间** 插入 `25`, 再删除 `30`

1.准备工作（创建链表）

```
#include <iostream>
#include <list>
using namespace std;

void printList(list<int>& lt) {
    for (int num : lt) {
        cout << num << " ";
    }
    cout << endl;
}

int main() {
    list<int> lt;
    // 初始化链表: 10 20 30 40
    lt.push_back(10);
    lt.push_back(20);
    lt.push_back(30);
    lt.push_back(40);
    printList(lt); // 输出: 10 20 30 40
}
```

2.第一步：获取迭代器，移动到中间位置

```
// 获取指向第一个元素的迭代器
list<int>::iterator it = lt.begin();

// 想指向 30 的位置（中间），需要向后移动2次
it++; // 指向 20
it++; // 指向 30
```

✅ 关键：只能用 ++/-- 一步步移动，不能写 `it+2`！

3. 第二步：中间插入 insert

```
// 在迭代器指向的【30 前面】插入 25
lt.insert(it, 25);
printList(lt); // 输出: 10 20 25 30 40
```

4. 第三步：中间删除 erase

```
// 此时 it 仍然指向 30，直接删除
lt.erase(it);
printList(lt); // 输出: 10 20 25 40
```


5. list 独有的神仙函数（链表专属！）

```
// 反转链表: 1 2 3 → 3 2 1
lt.reverse();

// 排序（默认升序）
lt.sort();

// 删除所有值为 10 的元素
lt.remove(10);
```

五、list 遍历方式（只有 2 种！）

因为不支持下标，所以不能用普通 for 循环！

方式 1：迭代器遍历（最标准）

```
for (list<int>::iterator it = lt.begin(); it != lt.end(); it++) {
    cout << *it << " ";
}
```

方式 2：范围 for（最简单）

```
for (int num : lt) {
    cout << num << " ";
}
```

六、完整示例代码（直接运行）

```
#include <iostream>
#include <list>
using namespace std;

void printList(list<int>& lt) {
    for (int num : lt) {
        cout << num << " ";
    }
    cout << endl;
}

int main() {
    list<int> lt;

    // 插入数据
    lt.push_back(10);
    lt.push_back(20);
    lt.push_front(5);
    printList(lt); // 5 10 20

    // 反转
```

```
lt.reverse();
printList(lt); // 20 10 5

// 排序
lt.sort();
printList(lt); // 5 10 20

// 删除元素
lt.remove(10);
printList(lt); // 5 20

system("pause");
return 0;
}
```

七、终极总结（背会这 4 点就够）

1. **list** = 双向链表，不连续内存
2. 最大优势：任意位置插入 / 删除速度极快
3. 最大缺点：不支持 `[]` 下标访问，只能迭代器 / 范围 `for` 遍历
4. 适用场景：需要频繁在中间增删数据

八、容器选择口诀（你学完 3 大序列容器啦！）

1. 只用尾部增删 → 选 `vector`
2. 两头都要增删 → 选 `deque`
3. 中间频繁增删 → 选 `list`

五、set 容器

`set` 是 STL 关联式容器，也是你接触的第一个用二叉树（红黑树）实现的容器！

它和你之前学的 `vector/list/deque` 完全不一样，天生自带两个超能力，这也是它唯一的核心：

set 容器 两大核心特性：去重、自排序

1. 所有元素自动排序（默认从小到大升序）
2. 所有元素唯一不重复（插入重复值会被自动忽略）

一句话：你往 `set` 里扔数据，它自动去重 + 排序，不用你写一行代码！

一、基础必备

1. 头文件

```
#include <set>    // 必须包含
using namespace std;
```

2. 重要规则

- 不支持 [] 下标访问 (和 list 一样)
- 不允许修改元素 (修改会破坏二叉树结构, 只能删了重新插)
- 查找速度极快 (比 vector 快得多)

二、最常用操作 (逐行教你)

1. 创建 set 容器

```
// 空 set
set<int> s1;

// 初始化列表 (C++11), 自动排序+去重
set<int> s2 = {3, 1, 2, 2, 3};
// 最终元素: 1,2,3 (重复的自动删掉, 顺序排好)
```

2. 插入元素 (唯一插入方式: insert)

set 只有 insert 插入, 插入自动排序 + 去重

```
set<int> s;

// 插入数据
s.insert(3);
s.insert(1);
s.insert(2);
s.insert(2); // 重复插入, 自动忽略
s.insert(1); // 重复插入, 自动忽略

// 最终 set 里的数据: 1 2 3
```

3. 遍历 set (只有 2 种方式)

set 不支持下标遍历, 只能用:

方式 1: 迭代器 (标准写法)

```
for (set<int>::iterator it = s.begin(); it != s.end(); it++) {  
    cout << *it << " ";  
}  
// 输出: 1 2 3 (自动排序)
```

方式 2: 范围 for (最简单)

```
for (int num : s) {  
    cout << num << " ";  
}
```

4. 删除元素 (erase)

```
set<int> s = {1,2,3,4,5};  
  
// 方式1: 按值删除 (最常用)  
s.erase(3); // 删除值为3的元素  
  
// 方式2: 按迭代器删除  
set<int>::iterator it = s.begin();  
s.erase(it); // 删除第一个元素  
  
// 方式3: 清空所有  
s.clear();
```

5. 查找元素 (find, set 最常用功能)

作用: 快速判断元素是否存在

核心语法:

```
// 语法  
set<类型>::iterator 迭代器名 = set对象.find(要查找的值);  
  
// 简化写法 (C++11 auto自动推导类型, 推荐新手用)  
auto it = s.find(目标值);
```

find 的两种返回结果:

find 只会返回两种迭代器, 这是判断是否找到的唯一标准:

1. 找到元素

返回指向该元素的迭代器, 解引用 `*it` 可以拿到元素值

2. 没找到元素

返回 `set对象.end()` → 尾后迭代器 (不指向任何有效元素, 代表查找失败)

最常用固定模板

```

set<int> s = {1,2,3};

// 查找值为2的元素，返回迭代器
set<int>::iterator it = s.find(2);

// 判断是否找到
if (it != s.end()) {
    cout << "找到元素: " << *it << endl;
} else {
    cout << "未找到" << endl;
}

```

6. 统计元素个数 (count)

因为 set 元素唯一，所以结果只有 0 或 1

- 0: 不存在
- 1: 存在

```

int num = s.count(2);
cout << num << endl; // 1

```

7. 大小 / 空判断

```

s.size();    // 获取元素个数
s.empty();  // 判断是否为空
s.swap(s2); // 交换两个set

```

三、完整可运行示例代码

```

#include <iostream>
#include <set>
using namespace std;

int main() {
    // 1. 创建set并插入数据（自动去重+排序）
    set<int> s;
    s.insert(5);
    s.insert(2);
    s.insert(8);
    s.insert(2); // 重复，无效
    s.insert(5); // 重复，无效

    // 2. 遍历
    cout << "set元素: ";
    for (int num : s) {
        cout << num << " ";
    }
    // 输出: 2 5 8
}

```

```

cout << endl;

// 3. 查找元素
if (s.find(5) != s.end()) {
    cout << "找到5" << endl;
}

// 4. 删除元素
s.erase(5);
cout << "删除后: ";
for (int num : s) {
    cout << num << " ";
}
// 输出: 2 8

system("pause");
return 0;
}

```

四、进阶：允许重复的 set → multiset

如果你需要重复元素，只需要自动排序，用 `multiset`：

```

#include <set>
multiset<int> ms;
ms.insert(2);
ms.insert(2);
ms.insert(1);
// 元素: 1,2,2 (允许重复, 自动排序)

```

用法和 `set` 完全一样，唯一区别：允许重复元素

五、set 对比你学过的容器（一眼看懂）

容器	排序	去重	下标访问	底层结构
vector	✗ 手动排	✗ 手动去重	✓ 支持	数组
list	✗ 手动排	✗ 手动去重	✗ 不支持	链表
set	✓ 自动排	✓ 自动去重	✗ 不支持	红黑树

六、set 适用场景（什么时候用？）

1. 需要自动给数据排序
2. 需要给数据去重
3. 需要快速查找某个元素是否存在

六、map容器

`map` 是 STL 最核心的关联式容器，和 `set` 是亲兄弟（底层都是红黑树），唯一的区别是：

- `set` 只存单个数据
- `map` 存键值对（key - value），也就是字典 / 映射关系

一、map 核心特性（死记这 6 条）

1. 存储键值对 `key-value`（比如 学号 -> 姓名，身份证 -> 信息）
 2. **key 唯一不重复**（重复 key 会覆盖 / 忽略）
 3. **支持 [] 下标访问**（`set` / `list` 都不支持，`map` 独有！）
 4. 底层红黑树，**查找速度极快** $O(\log n)$
 5. **key 不能改，value 可以随便改**
 6. **key 自动排序**（默认从小到大升序），和 **value 半毛钱关系没有**
- **key 必须支持“比较大小”**，否则无法排序（编译报错）
 - `int`、`float`型比较数值大小
 - `char`比较ASCII码大小
 - `string`**按「字典序」排序**（和英语词典排序一模一样）

1. 逐字符比较 ASCII 码
2. 前面字符相同，比下一个
3. 短字符串 < 长字符串（前缀相同时）

- **自定义数据类型（重点！坑点！）**

比如你自己写的 `Person` 类、`Student` 类

✗ 错误：直接用自定义类当 key → 编译报错！

因为编译器不知道如何比较两个自定义对象！

✓ 正确：必须重载 < 运算符

告诉 `map` 按照什么规则排序

完整示例代码

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

// 自定义类
class Person {
```

```

public:
    string name;
    int age;
    Person(string n, int a) : name(n), age(a) {}
};

// 必须重载 < 运算符!!!
bool operator<(const Person& p1, const Person& p2) {
    // 规则: 按年龄从小到大排序
    return p1.age < p2.age;
}

int main() {
    // 自定义类作为 key
    map<Person, int> m;
    m[Person("李四", 20)] = 1;
    m[Person("张三", 18)] = 2;

    // 自动按年龄排序: 18 → 20
    return 0;
}

```

如何修改排序规则 (从大到小 降序)

默认是升序, 想降序, 只需要加一个参数: `std::greater<T>`

示例 (int 降序)

```

#include <map>
// 第三个参数: greater 表示降序
map<int, string, greater<int>> m;
m[1] = "张三";
m[3] = "李四";
m[2] = "王五";

// 排序结果: 3 → 2 → 1

```

二、必备基础

1. 头文件

```

#include <map>    // 必须包含
using namespace std;

```

2. 键值对 pair (map 的元素)

map 里的每一个元素, 都是一个 pair 对组

- `pair<T1, T2>`: T1=key (键), T2=value (值)
- `pair.first` → 访问 key
- `pair.second` → 访问 value
- 创建对组: `make_pair(key, value)`

三、map 最常用操作

1. 创建 map 容器

语法: `map<key类型, value类型> 变量名`

```
// 空map: key=int, value=string
map<int, string> m;

// 直接初始化
map<int, string> m2 = { {1, "张三"}, {2, "李四"} };
```

2. 插入数据 (2 种最常用方式)

方式 1: `insert + make_pair` (推荐, 安全)

```
map<int, string> m;
m.insert(make_pair(1, "孙悟空"));
m.insert(make_pair(2, "韩信"));
m.insert(make_pair(3, "姐己"));
```

方式 2: `[]` 直接赋值 (最简单, 最常用)

- `[]`里面输入key的名字, 如果是字符串就是`["key"]`

```
m[4] = "王昭君"; // key=4, value=王昭君
m[5] = "赵云";
```

✅ 重点: 如果 key 已存在, `[]` 会覆盖原有 value

3. 访问 / 修改 value (map 最爽的功能)

`[]`里面输入key的名字, 如果是字符串就是`["key"]`

```
// 1. [] 访问 (最常用)
cout << m[1] << endl; // 输出: 孙悟空

// 2. 修改 value (key 不能改!)
m[1] = "齐天大圣"; // 把key=1的value改成齐天大圣
```

4. 查找元素 `find` (和 set 一模一样)

- 传入 key 查找
- 找到: 返回指向该 pair 的迭代器

所以不能用*it, 而是it->first和it->second

- 没找到: 返回 `m.end()`

```
// 查找 key=2 的元素
auto it = m.find(2);

if (it != m.end()) {
    cout << "找到元素: ";
    cout << "key=" << it->first;    // 迭代器用 -> 访问
    cout << " value=" << it->second << endl;
} else {
    cout << "未找到" << endl;
}
```

5. 遍历 map (2 种方式)

map 遍历的是 `pair`，必须用 `first/second` 取值

方式 1: 迭代器遍历 (标准)

```
for (map<int, string>::iterator it = m.begin(); it != m.end(); it++) {
    cout << "key:" << it->first << " value:" << it->second << endl;
}
```

方式 2: 范围 for (最简单)

```
for (auto& p : m) {    // p 就是 pair
    cout << p.first << " : " << p.second << endl;
}
```

6. 删除元素 `erase`

```
map<int, string> m = {{1, "张三"}, {2, "李四"}, {3, "王五"}};

// 方式1: 按 key 删除 (最常用)
m.erase(2);    // 删除 key=2 的元素

// 方式2: 按迭代器删除
auto it = m.find(3);
m.erase(it);

// 清空所有
m.clear();
```

7. 统计 / 大小 / 判空

```
m.size();    // 获取元素个数
m.empty();   // 是否为空
m.count(1);   // 统计key=1的个数 (0或1, 和set一样)
```

四、完整可运行示例代码

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    // 1. 创建map: key=int学号, value=string姓名
    map<int, string> m;

    // 2. 插入数据
    m.insert(make_pair(1, "孙悟空"));
    m.insert(make_pair(2, "韩信"));
    m[3] = "姐己";
    m[4] = "王昭君";

    // 3. 遍历map
    cout << "全部数据: " << endl;
    for (auto& p : m) {
        cout << "学号: " << p.first << " 姓名: " << p.second << endl;
    }

    // 4. 查找key=2
    auto it = m.find(2);
    if (it != m.end()) {
        cout << "\n查找成功: " << it->second << endl;
    }

    // 5. 修改value
    m[1] = "齐天大圣";
    cout << "\n修改后: " << m[1] << endl;

    // 6. 删除key=3
    m.erase(3);
    cout << "\n删除后遍历: " << endl;
    for (auto& p : m) {
        cout << p.first << " : " << p.second << endl;
    }

    system("pause");
    return 0;
}
```

五、进阶：允许重复 key → multimap

- 和 map 唯一区别：key 可以重复
- 不支持 [] 访问
- 用法和 map 完全一致

```
#include <map>
multimap<int, string> mm;
mm.insert({1, "张三"});
mm.insert({1, "李四"}); // key重复, 允许!
```

六、map vs set（一眼区分）

容器	存储数据	key 唯一性	排序	[] 访问	底层
set	单个值	元素唯一	是	✗ 不支持	红黑树
map	键值对	key 唯一	是	✓ 支持	红黑树

七、map 使用场景（什么时候用？）

1. 需要存储**对应关系**（学号 - 姓名，ID - 信息）
2. 需要**快速通过 key 查找 value**
3. 需要**自动按 key 排序**

七、unordered_map

你已经学会了 `map`，`unordered_map` 1 分钟就能上手！

它俩用法几乎完全一样，唯一的区别就是：

✓ `map`：有序（红黑树）、慢一点

✓ `unordered_map`：无序（哈希表）、超快 $O(1)$ 、LeetCode 必用神器

一、核心特性（死记 3 条）

1. 存储**键值对** `key-value`（和 `map` 一模一样）
2. **key 唯一不重复**（和 `map` 一模一样）
3. **不自动排序！**（和 `map` 最大区别）
4. **查找 / 插入 / 删除速度极快**（刷题永远优先用它）

二、头文件（必须加）

```
#include <unordered_map> // 不是map! 单独头文件
using namespace std;
```

三、常用 API（和 map 完全通用，直接照搬）

我直接给你刷题最常用的 5 个操作，复制就能用：

1. 创建哈希表

```
// key: int（比如学号），value: int（比如成绩）
unordered_map<int, int> hash_map;

// 常用：字符串作为key
unordered_map<string, int> str_map;
```

2. 插入数据（2 种方式）

```
// 方式1: [] 赋值（最简单，刷题首选）
hash_map[1] = 100;
hash_map[2] = 200;

// 方式2: insert 插入（和map一样）
hash_map.insert(make_pair(3, 300));
```

3. 访问 / 修改数据

```
// 用 [] 直接取值/改值
cout << hash_map[1] << endl; // 取值
hash_map[1] = 99;             // 修改
```

4. 查找元素（LeetCode 最常用）

和 map/set 的 find 用法完全一样：

```
// 查找 key=2
auto it = hash_map.find(2);

// 判断是否找到
if (it != hash_map.end()) {
    cout << "找到: key=" << it->first << " value=" << it->second << endl;
} else {
    cout << "未找到" << endl;
}
```

5. 遍历元素（和 map 一模一样）

```
// 范围for（刷题首选，最简单）
for (auto& p : hash_map) {
    cout << p.first << " : " << p.second << endl;
}
```

6. 删除元素

```
hash_map.erase(2); // 删除key=2的元素
hash_map.clear();  // 清空
```

四、unordered_map vs map（一眼看懂）

容器	底层	排序	速度	刷题选择
map	红黑树	自动有序	慢	几乎不用
unordered_map	哈希表	无序	超快	必用！

五、LeetCode 必用场景（直接套）

- 1. 两数之和（最经典）
- 2. 统计字符出现次数
- 3. 快速判断元素是否存在
- 4. 记录映射关系

六、完整可运行代码（直接复制）

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main() {
    // 1. 创建哈希表
    unordered_map<int, string> hash_map;

    // 2. 插入数据
    hash_map[1] = "张三";
    hash_map[2] = "李四";
    hash_map[3] = "王五";

    // 3. 遍历（无序输出！顺序随机）
    cout << "遍历结果: " << endl;
    for (auto& p : hash_map) {
        cout << p.first << " : " << p.second << endl;
    }

    // 4. 查找
    auto it = hash_map.find(2);
    if (it != hash_map.end()) {
        cout << "\n查找到: " << it->second << endl;
    }

    // 5. 修改
    hash_map[1] = "张三三";
```

```
system("pause");  
return 0;  
}
```

七、总结（背这一句就够）

`unordered_map` = 无序、超快版的 `map`

用法和 `map` 完全一样，刷 LeetCode 永远用它代替 `map`！

八、unordered_set

本质

底层是哈希表的无序集合，和 `set` 规则一致，但更快、无序。

二、3 大核心特性（死记）

1. 只存单个值（不是键值对，和 `set` 一样）
2. 元素唯一，自动去重（重复插入会被忽略）
3. 无序、查找 / 插入速度极快 $O(1)$ （刷题首选）
4. 不支持 `[]`，不能修改元素

三、头文件

```
#include <unordered_set> // 专用头文件  
using namespace std;
```

四、🔥 刷题必备 API（和 set 完全一模一样）

直接照搬 `set` 的用法，不用学新东西！

1. 创建哈希集合

```
unordered_set<int> uset; // 存int  
unordered_set<string> uset2; // 存字符串
```

2. 插入元素（自动去重）

```
uset.insert(10);
uset.insert(20);
uset.insert(10); // 重复，自动忽略
```

3. 查找元素（最常用！判断值是否存在）

```
// 查找 20
auto it = uset.find(20);
if (it != uset.end()) {
    cout << "找到元素" << endl;
}
```

4. 删除元素

```
uset.erase(20); // 按值删除
uset.clear();   // 清空
```

5. 遍历（2 种方式）

```
// 范围for（刷题首选）
for (int num : uset) {
    cout << num << " ";
}

// 迭代器
for (auto it = uset.begin(); it != uset.end(); it++) {
    cout << *it << " ";
}
```

6. 大小 / 判空

```
uset.size(); // 元素个数
uset.empty(); // 是否为空
```

五、完整可运行代码

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main() {
    // 1. 创建
    unordered_set<int> uset;

    // 2. 插入（自动去重）
    uset.insert(3);
    uset.insert(1);
```



```
uset.insert(2);
uset.insert(3);

// 3. 遍历（无序输出）
cout << "元素: ";
for (int num : uset) {
    cout << num << " ";
}

// 4. 查找
if (uset.find(2) != uset.end()) {
    cout << "\n找到2";
}

system("pause");
return 0;
}
```

六、一眼区分四兄弟（永不混淆）

容器名称	中文名	存储内容	自动排序	底层结构	查询速度	刷题选择
set	有序集合	单个值	✔ 有序 (升序)	红黑树	$O(\log n)$	✘ 几乎不用
unordered_set	无序哈希集合	单个值	✘ 无序	哈希表	$O(1)$ 超快	✔ 首选
map	有序映射	键值对 (key-value)	✔ 有序 (按 key)	红黑树	$O(\log n)$	✘ 几乎不用
unordered_map	无序哈希映射	键值对 (key-value)	✘ 无序	哈希表	$O(1)$ 超快	✔ 首选

七、LeetCode 必用场景

- 1. 快速判断一个数是否出现过
- 2. 数组 / 字符串去重
- 3. 查重、找重复元素

九、stack容器适配器

stack 中文叫栈，是 STL 的容器适配器（不是底层容器，是封装好的工具），它的规则死记一条就够：

🔥 核心规则：先进后出（FILO）

就像装子弹的弹夹、叠盘子：

- 最先放进去的，压在最底下，最后才能拿出来
- 最后放进去的，在最顶上，最先拿出来
- 只能访问、操作最顶端的元素，不能访问中间 / 底部元素

一、stack 核心特性（必背）

1. 先进后出
2. 只能操作栈顶（没有中间访问、没有遍历）
3. 不支持迭代器、不支持 [] 下标
4. 底层默认用 deque 实现，你不用管底层
5. 接口极少，是 STL 里最好学的容器

二、基础用法

1. 头文件

```
#include <stack> // 必须包含
using namespace std;
```

2. 创建栈

```
// 创建一个存储 int 类型的空栈
stack<int> st;

// 也可以存其他类型：string、自定义对象等
stack<string> st2;
```

三、🔥 stack 只有 5 个常用 API（全背会，够用一生）

函数	作用	口诀
push(值)	入栈（向栈顶添加元素）	压栈
pop()	出栈（删除栈顶元素）	弹栈
top()	获取栈顶元素	取顶
empty()	判断栈是否为空（空返回 true）	判空
size()	获取栈中元素个数	大小

四、超级重点（新手必看）

✗ `pop()` 只删除，不返回值！

✓ 想拿栈顶元素，必须先用 `top()`，再用 `pop()` 删除

```
// 错误写法：想直接拿到删除的元素
int val = st.pop(); // 报错！pop 没有返回值

// 正确写法：先取栈顶，再删除
int val = st.top(); // 拿到栈顶
st.pop();           // 删除栈顶
```

五、完整可运行代码（一看就懂）

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    // 1. 创建栈
    stack<int> st;

    // 2. 入栈（压栈）
    st.push(10); // 栈顶
    st.push(20);
    st.push(30); // 最后入栈，在最顶端

    // 3. 查看栈顶元素
    cout << "当前栈顶: " << st.top() << endl; // 输出 30

    // 4. 查看元素个数
    cout << "栈大小: " << st.size() << endl; // 输出 3

    // 5. 出栈（删除栈顶）
    st.pop(); // 删除 30
    cout << "出栈后栈顶: " << st.top() << endl; // 输出 20

    // 6. 循环清空栈（先进后出）
    while (!st.empty()) {
        cout << st.top() << " "; // 20 10
        st.pop();
    }

    system("pause");
    return 0;
}
```

运行结果

```
当前栈顶: 30
栈大小: 3
出栈后栈顶: 20
20 10
```

六、stack 为什么不能遍历 / 访问中间元素？

因为栈的设计初衷就是只允许操作栈顶，这是它的规则！

如果需要遍历、访问中间元素，说明你用错容器了，应该用 `vector` / `list`。

七、stack 适用场景

1. 撤销操作 (Ctrl+Z)
2. 函数调用栈 (系统底层)
3. 括号匹配检查
4. 浏览器后退 / 前进
5. 弹夹、叠盘子这类先进后出的场景

八、终极总结（背会这 3 句）

1. **stack = 栈 = 先进后出**
2. 只能操作栈顶: `push` 入栈、`top` 取值、`pop` 删除
3. **无迭代器、无下标、不遍历**，是 STL 最简单的容器

十、queue 容器适配器

你刚学会了 **stack (栈, 先进后出)**，`queue` 就是它的亲兄弟——**队列**，规则完全相反，是 STL 第二简单的容器，零学习成本！

核心规则（死记这一句）

queue = 队列 = 先进先出 (FIFO)

就像排队买票、超市结账：

- 最先进入队列的人，最先出去
- 最后进入的人，最后出去
- 只能操作队头 (出去) *和*队尾 (进来)，不能访问中间元素

一、queue 核心特性

- 1. 先进先出 (FIFO) (和 stack 完全相反)
- 2. 容器适配器，底层默认用 deque 实现，不用管内存
- 3. 不支持迭代器、不支持 [] 下标、不能遍历中间元素
- 4. API 极少，和 stack 几乎一一对应

二、基础用法

1. 头文件

```
#include <queue> // 必须包含
using namespace std;
```

2. 创建队列

```
// 空队列，存储int类型
queue<int> q;

// 可存储任意类型：string、自定义对象
queue<string> q2;
```

三、🔥 queue 6 个常用 API（全背会，通用所有场景）

和 stack 对比记忆，一眼区分：

操作	queue (队列)	作用	stack (栈) 对应
入队	push(值)	队尾插入元素	push (栈顶入栈)
出队	pop()	删除队头元素 (无返回值)	pop (栈顶出栈)
取值	front()	获取队头元素	top() (栈顶)
取值	back()	获取队尾元素	无
判断	empty()	队列是否为空	一样
大小	size()	元素个数	一样

⚠️ 超级重点（和 stack 一模一样）

pop() 只删除、不返回值！

想获取队头元素 → 必须先 front()，再 pop()

```
// 错误
int val = q.pop();

// 正确
int val = q.front(); // 先拿队头
q.pop();           // 再删除
```

四、完整可运行代码（最直观）

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    // 1. 创建队列
    queue<int> q;

    // 2. 队尾入队（添加元素）
    q.push(10); // 第一个进
    q.push(20);
    q.push(30); // 最后一个进

    // 3. 查看队头、队尾
    cout << "队头: " << q.front() << endl; // 10
    cout << "队尾: " << q.back() << endl;   // 30
    cout << "元素个数: " << q.size() << endl; // 3

    // 4. 队头出队（删除元素）
    q.pop(); // 删除 10
    cout << "出队后, 新队头: " << q.front() << endl; // 20

    // 5. 循环清空队列（先进先出）
    while (!q.empty()) {
        cout << q.front() << " "; // 20 30
        q.pop();
    }

    system("pause");
    return 0;
}
```

运行结果

```
队头: 10
队尾: 30
元素个数: 3
出队后, 新队头: 20
20 30
```

五、stack vs queue 终极对比（永不混淆）

容器	全称	规则	操作位置	取值函数
stack	栈	先进后出	仅栈顶	top()
queue	队列	先进先出	队头 + 队尾	front() / back()

六、queue 适用场景

- 1. 排队系统（食堂、银行、取票）
- 2. 消息队列、任务调度
- 3. 广度优先搜索（BFS，算法常用）
- 4. 按顺序处理的任务

七、终极总结（背会这 3 点）

- 1. queue = 队列 = 先进先出
- 2. 核心操作：push（队尾入）、front（取队头）、pop（删队头）
- 3. 无迭代器、无下标、不遍历，和 stack 一样简单

十一、priority_queue

一、核心定义

中文名

优先队列 / 堆

核心规则

不遵守先进先出！队头永远是「优先级最高」的元素

- ✔ 默认：大顶堆 → 数值最大的元素在队头
- ✔ 可修改：小顶堆 → 数值最小的元素在队头

二、基础信息

- 1. 头文件：和普通队列完全一样

```
#include <queue>
using namespace std;
```

- 1. 容器适配器：复用 `vector` 作为底层，不用管内存
- 2. 只能操作队头：和 `stack/queue` 一样，不能遍历中间元素

三、🔥 常用 API（和普通 `queue` 几乎一模一样）

函数	作用	关键点
<code>push(val)</code>	插入元素	自动排序，调整堆结构
<code>pop()</code>	删除队头	删除优先级最高的元素
<code>top()</code>	获取队头	获取最大值 / 最小值
<code>empty()</code>	判断是否为空	
<code>size()</code>	获取元素个数	

⚠️ 和 `queue` 唯一区别：取值用 `top()`，不是 `front()`！

四、🔥 刷题必考：大顶堆 / 小顶堆

1. 默认大顶堆（最常用）

最大值永远在队头

```
// 语法: priority_queue<类型> 变量名
priority_queue<int> pq;
```

2. 手动创建小顶堆（TopK 问题必用）

最小值永远在队头，固定写法，直接背

```
// 固定三参数：类型 + 底层容器 + 比较规则
priority_queue<int, vector<int>, greater<int>> pq;
```

五、完整可运行代码（大顶堆 + 小顶堆）

示例 1：默认大顶堆

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    // 大顶堆：最大值优先
    priority_queue<int> pq;

    // 插入数据
    pq.push(5);
```



```

pq.push(2);
pq.push(8);
pq.push(1);

// 队头永远是最大值 8
cout << "队头最大值: " << pq.top() << endl;

// 依次出队: 8 → 5 → 2 → 1
while (!pq.empty()) {
    cout << pq.top() << " ";
    pq.pop();
}

system("pause");
return 0;
}

```

示例 2：小顶堆（固定写法）

```

#include <iostream>
#include <queue>
using namespace std;

int main() {
    // 小顶堆：最小值优先（固定写法）
    priority_queue<int, vector<int>, greater<int>> pq;

    pq.push(5);
    pq.push(2);
    pq.push(8);
    pq.push(1);

    // 队头最小值: 1
    cout << "队头最小值: " << pq.top() << endl;

    // 依次出队: 1 → 2 → 5 → 8
    while (!pq.empty()) {
        cout << pq.top() << " ";
        pq.pop();
    }

    system("pause");
    return 0;
}

```

六、普通队列 vs 优先队列（一眼区分）

容器	规则	队头元素	底层
queue	先进先出	最早插入的元素	deque
priority_queue	按优先级排序	最大 / 最小值	vector（堆结构）

七、LeetCode Hot100 必考场景

1. 前 K 个高频元素 (TopK 问题)
2. 合并 K 个升序链表
3. 数据流中的中位数
4. 贪心算法、堆排序

八、终极总结 (背这 3 句)

1. `priority_queue` = 优先队列 = 堆
2. 默认大顶堆 (最大值优先) , 小顶堆背固定写法
3. 核心操作: `push` 插入、`top` 取值、`pop` 删除